

Chapter 7 - Key Management and Distribution

I. Symmetric Key Distribution using Symmetric Encryption

Key Distribution: In symmetric encryption, the two parties share the same key to an exchange the message, and that key must be protected from access by others. The strength of any cryptographic system rests with the key distribution technique, a term that refers to delivering a key to two parties who want to exchange data, without allowing others to see the key. For two parties Bob and Alice want to communicate, key distribution can be achieved in a number of ways, as follows:

- Bob can select the key and deliver it to Alice.
- A third party C can select the key and physically deliver it to Bob and Alice.
- If Bob and Alice have previously and recently used a key, one party can transmit the new key to the other, encrypted using the old key.
- If Bob and Alice has an encrypted connection to a third party C, C can deliver a key on the encrypted links to Bob and Alice.
- In symmetric encryption is number of user is N then total [n(n-1)]/2 number of keys are required.
- If we use 4th option of key distribution the N numbers of keys only are required.
- Session key
- Master key

II. Key Distribution Scenario

- Let us assume that Bob wishes to establish a logical connection with Alice and requires a one-time session key to protect the data transmitted over the connection.
- A has a master key, Ka, known only to itself and the KDC; similarly, Alice shares the master key, Kb with the KDC. Bob issues a request to KDC for a session key to protect a logical connection to Alice.
- The message includes the identity of Bob and Alice and unique identifier, N1, for this transaction, which is refer to as **nonce**.
- The nonce may be a timestamp, counter, or a random number; the minimum requirement is that it differs with each request.



- It prevent masquerade, it should be difficult for an opponent to guess the nonce.
- Thus, a random number is a good choice for a nonce.

Hierarchical Key Control

- It is not necessary to limit the key distribution function to a single KDC.
- For very large network, it may not be practical to do so.
- As an alternative, a hierarchy of KDCs can be established.
- For an example, there can be local KDCs, each responsible for a small domain of the overall internetworking, such as single LAN or a single building.
- For communication among entities within the same local domain, the local KDC responsible for key distribution.
- If two entities in different domains desire a shared key, then the corresponding local KDC s can communicate through a global KDC.
- Such a scheme limit the damage of a faulty KDC to its local area only.

Session Key Lifetime

- The more frequently session keys are exchanged, the more secure they are.
- On the other hand, the distribution of session keys delays the start of exchange and place a burden on network capacity.
- A security manager must try to balance these completing consideration in determining the lifetime of a particular session key.
- For connection-oriented protocols, one obvious choice is to use the same session key for the length of time that the connection is open, using new session key for each new session.
- For connectionless protocols, such as transaction-oriented protocols, the most secure approach is to use a new session key for each exchange.
- However, this negates one of the principal benefits of connectionless protocols, which is minimum overhead and delays for each transaction.
- A better strategy is to use a given session key for a certain fixed period only or for a certain number of transactions.

A Transparent Key Control Scheme

- The scheme is useful is useful for providing end-to-end encryption at a network or transport level in a way that is transparent to the end user.

- The approach assumes that communication makes use of a connection-oriented end-to-end protocol, such as TCP.
- When one host wishes to set up a connection to another host, it transmits a connection request packet.
- The session security module (SSM) saves that packet and applies to the KDC for permission to establish the connection.
- The communication between the SSM and the KDC is encrypted using the master key shared only by SSM and the KDC.
- If the KDC approves the connection request, it generates the session key and delivers it to the two appropriate SSMs, using a unique permanent key for each SSM.
- The requesting SSM can now release the connection request packet, and a connection is set up between the two end systems.

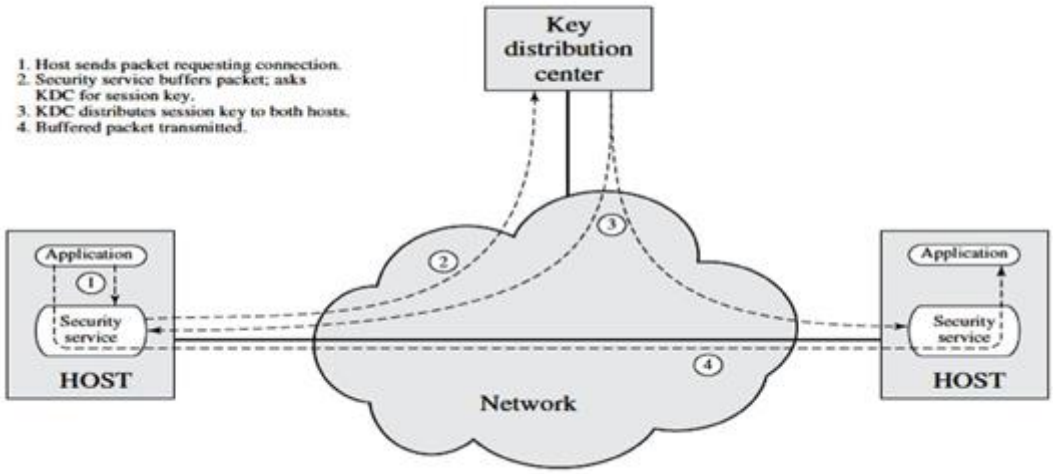


Figure 7.1 Automatic Key Distributions for Connection-Oriented Protocol

Decentralized Key Control

- A session key may be established with the following sequences of the steps.
- A issues a request to B for a session key and includes a nonce, N1.
- B response with a message that is encrypted using the shared master key. The response includes the session key selected by B, the value f(N1), and another nonce N2.
- Using the new session key, A returns f(N2) to B.
- Because the message transferred using the master key are short, cryptanalysis is difficult.

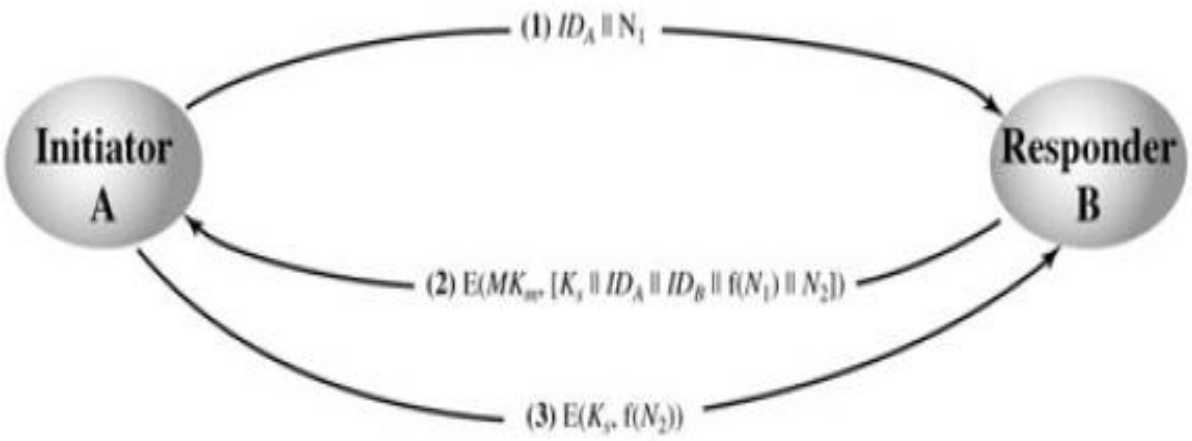


Figure 7.2 Decentralized Key Control

Symmetric Key Distribution using Asymmetric Encryption

Once the public key is shared or become accessible, secure communication that thwarts both eavesdropping and tampering are possible. Encrypted Public key provides for the sharing of secret keys i.e. is used for conventional encryption.

- **Simple Secret Key Distribution**

If A and B are communicates, the following procedure is employed:

1. A create a public/private key pair {*PUa, PRa}* and send a message to B consisting of PUa and an identifier of A, IDA.
2. B create a secret key, Ks, and send it to A, encrypted with A's public key.
3. A computes D(PRa, E(PUa, Ks)) to recover the secret key. If A can decode the letter, only A and B can know the identity of Ks.
4. A remove PUa and PRa and B remove PUa.

Now, A and B can securely communicates using conventional encryption and the session key Ks. After completion of the exchange, both A and B remove Ks. Now, no keys exist before the start of the communication and none exist after the completion

of communication. Hence, the minimal risk of compromise the keys.

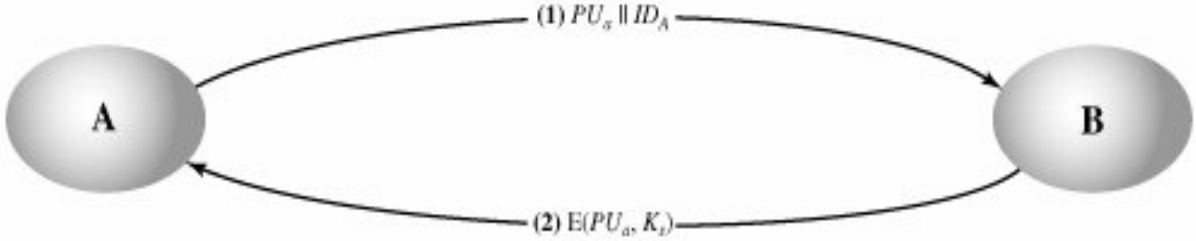


Figure 7.3 Simple Secret Key Distribution [1]

- **Secret Key Distribution with Confidentiality and Authentication**

In[NEED78], this approach provides protection against both active and passive attacks. Assuming A and B have exchanged public keys:

1. A uses B's public key to encrypt a message to B containing an identifier of A (IA) and a nonce (N1), which is used to identify this transaction uniquely.
2. B sends a PUa-encoded message to A containing A's nonce (N1) as well as a new nonce created by B (N2) Because only B could have decrypted message(3), the presence of N1 in message(6) assures A that B is the correspondent.
3. A returns N2, encrypted using B's public key, to assure B that its correspondent is A.
4. A chooses Ks for a hidden key and sends M = E(PUb, E(PRa, Ks)) to B. Whe n B's public key is encrypted, only B can read it; encryption with the private ke y of A means that only A can transmit it.
5. For recovery the secret key, B computes D(PUa, D(PRb, M.

Hence, this scheme assured both confidentiality and authentication in the exchange of a secret key.

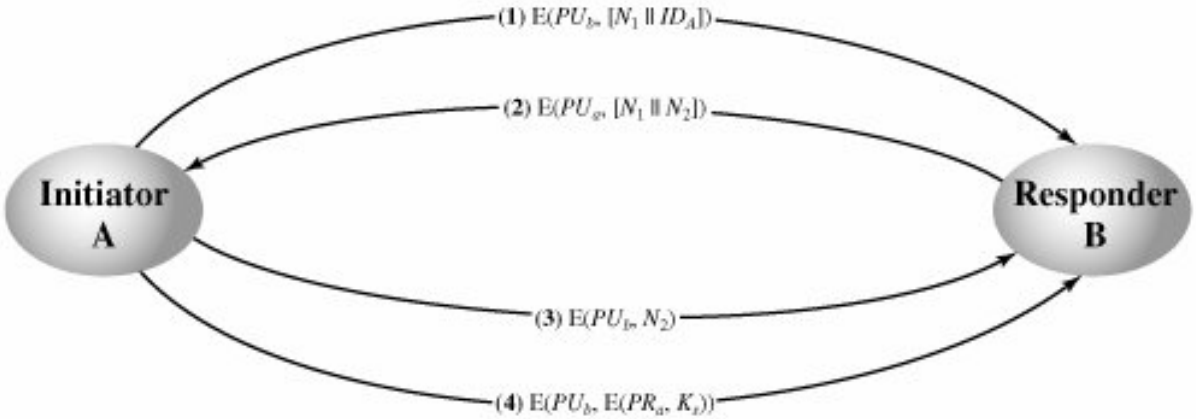


Figure 7.4 Secret Key Distributions with Confidentiality and Authentication [3]

Distribution of public key

Several technique have been proposed for the distribution of public key which can mostly be grouped into the catagories shown:

- Public announcement
- Publicly available directory
- Public-key authority
- **Public-key certificates**

Public announcement:



Figure 7.5 Uncontrolled Public-Key Distributions [3]

- The point of public-key encryption is that the public key is public, so any participant can give any other participant his or her public key or broadcast the key to the entire world.
- The main weakness is forgery, anyone can build a key claiming to be someone else and broadcast it, and they can masquerade as the claimed user before the forgery is discovered.

Publicly available directory:

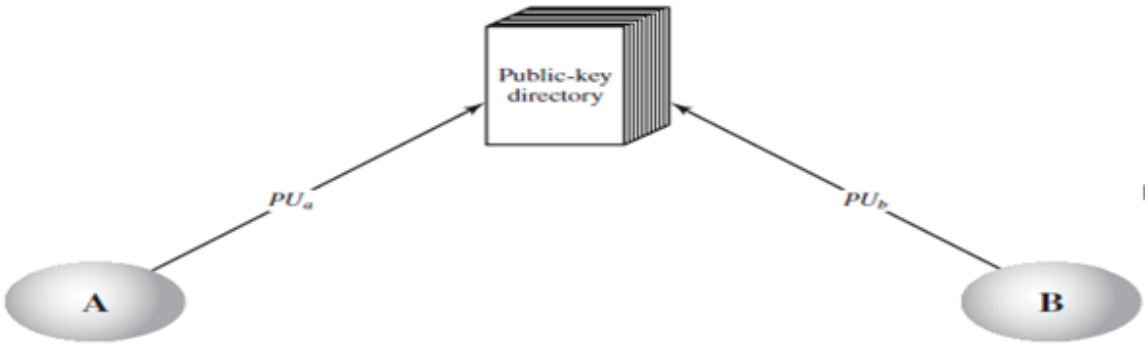


Figure 7.6 Public-Key Publications

- can obtain greater security by registering keys with a public directory
- directory must be trusted with properties:
  - contains {name,public-key} entries
  - participants register securely with directory
  - participants can replace key at any time
  - directory is periodically published
  - directory can be accessed electronically
- still vulnerable to tampering or forgery

Public-key authority

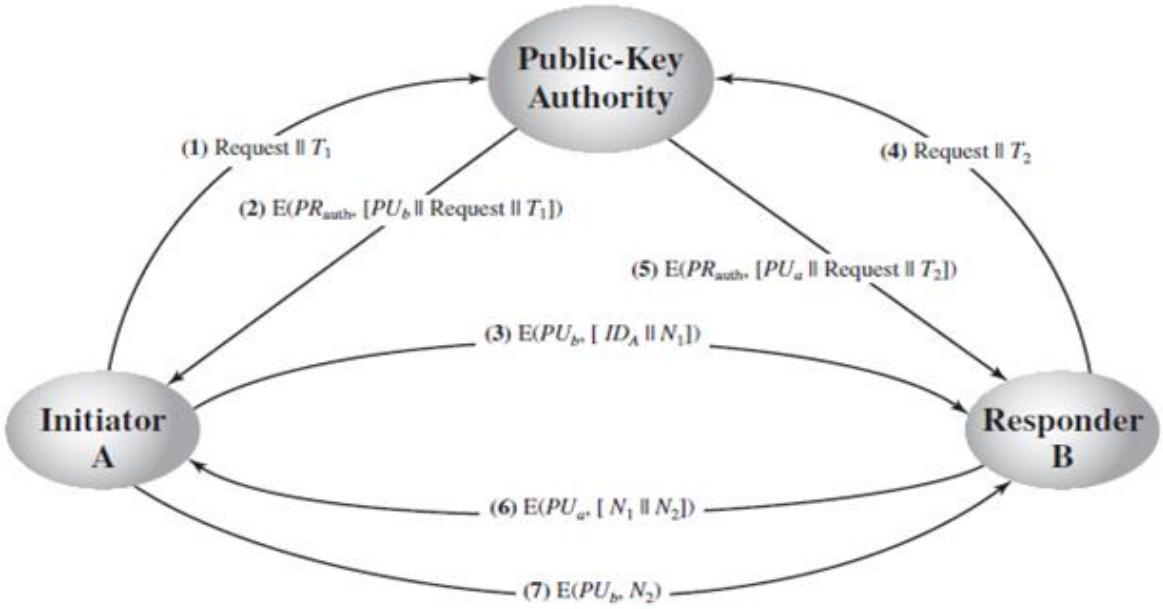


Figure 7.7 Public-key Distribution Scenario [6]



- Stronger security for public-key distribution can be achieved by providing tighter control over the distribution of public keys from the directory.
- It requires users to know the public key for the directory, and that they interact with directory in real-time to obtain any desired public key securely.
- Totally seven messages are required:
  - A sends a times tamped message to the public-key authority containing a request for the current public key of B.
  - The authority responds with a message that is encrypted using the authority's private key, PR_{auth} Thus, A is able to decrypt the message using the authority's public key. Hence, A is ensured that the originated of message with the authority. The message includes the following:
    - B's public key, PU_b which A can use to encrypt messages destined for B.
    - The original request, to enable A to match this response with the corresponding earlier request and to verify that the original request was not altered before reception by the authority.
    - The original timestamp, A can determine that this is not an old message from the authority containing a key other than B's current public key.
  - A stores B's public key and also uses it to encrypt a message to B containing an identifier of A (ID_A) and a nonce (N_1), which is used to identify this transaction uniquely.
  - B retrieves A's public key from the authority in the same manner as A retrieved B's public key.
  - At this point, public keys have been securely delivered to A and B, and they may begin their protected exchange. However, two additional steps are desirable:

- B sends a message to A encrypted with PU_a and containing A's nonce

kingraja@gmail.com

(N_1) as well as a new nonce generated by B (N_2) Because only B could have decrypted message (3), the presence of N_1 in message (6) assures A that the correspondent is B.

- To assure B that its correspondent is A, A returns N2, encrypted using B's public key.

Public-key certificates

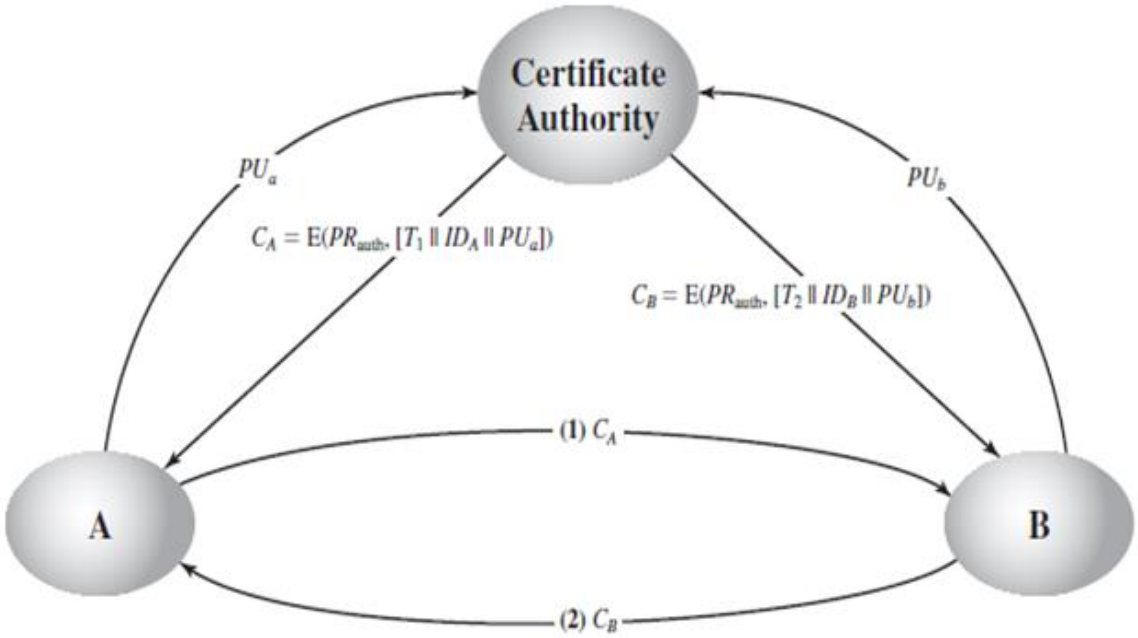


Figure 7.8 Exchange of Public-Key Certificates [6]

- With any other user they wish to contact, a user must apply to the authority for a public key and it is also prone to tampering.
- Public Key Certificates may be used without contacting a public key authority to exchange keys.
- A certificate connects a public key identity with all content signed by a trusted Public Key or Certificate Authority.
- This can be verified by anyone who knows the public-key authorities public-key.
- A participant can also convey its key information to another by transmitting its certificate.
- Other participants will check that the authority created the certificate. On this device, we can put the following specifications:
- Any participant can read a certificate to determine the name and public key of the certificate's owner.

- Any participant can verify that the certificate originated from the certificate authority and is not counterfeit.
- Only the certificate authority can create and update certificates.
- Any participant can verify the currency of the certificate.

For formatting public certificates, one scheme has been widely accepted: the X.509 standard.

X.509 certificates

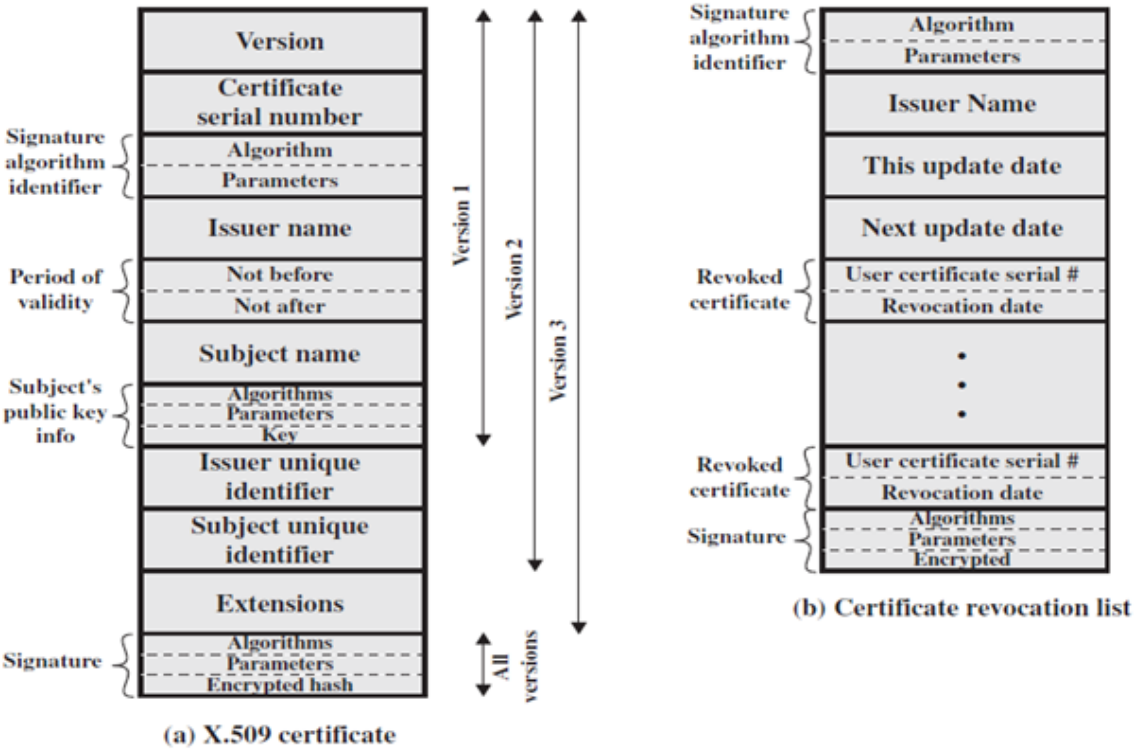


Figure 7.9 X.509 Formats [7]

- The X.509 Directory Authentication Service recommendation from ITU-T is part of the X.500 set of recommendations that describe a directory service.
- • X.509 describes a system for delivering authentication services to its users via the X.500 directory.
- The directory may serve as a repository of Public Certificates.
- Each certificate includes a user's public key and is signed with a Trusted Certificate Authority private key.
- The standard uses the notation for a certificate of:



CA<<A>> where the CA signs the certificate for user A with its private key. In more detail CA<<A>> = CA {V, SN, AI, CA, UCA, A, UA, Ap, TA}

If the corresponding public key is known to a user, then that user can verify that a certificate signed by the CA is valid.

- **Version:** It differentiates between successive certificate format versions; version 1 is the default version. Whether a unique identifier of the issuer or a unique identifier of the topic is present, the value must be Version 2. The version must be version 3 if one or more extensions are present.
- **Serial number:** An integer value unique within the issuing CA that is unambiguously associated with this certificate.
- **Signature algorithm identifier:** The algorithm that is used to sign the certificate and any relevant parameters. As this information is replicated at the end of the certificate in the signature field, this field has little if any, usefulness.
- **Issuer name:** X.500 is the name of the CA that created and signed this certificate.
- **Period of validity:** Consists of two dates: the first and last on which the certificate is valid.
- **Subject name:** The name of the individual that this certificate applies to. That is, the public key of the subject holding the corresponding private key is approved by this certificate.
- **Subject's public-key information:** The subject's public key, plus an algorithm identifier for which this key is to be used, along with any relevant parameters.
- **Issuer unique identifier:** An optional-bit string field that is used to uniquely define the issuing CA if the X.500 name has been reused for various entities.
- **Subject unique identifier:** An optional-bit string field that is used to uniquely define the topic in the event that various entities have reused the X.500 tag.
- **Extensions:** A set of one or more fields for extension. In version3, extensions were introduced and are addressed later in this section.
- **Signature:** It includes all the other certificate fields; it contains the hash code of the other fields encrypted by the private key of the CA. This area contains the identifier for the signature algorithm.

In version2, the unique identifier fields were introduced to handle the potential reuse over time of subjectand/or issuer names. Rarely are these fields included. For defining a credential, the standard uses the following notation:



CA<<A>> = CA {V, SN, AI, CA, UCA, A, UA, Ap, TA}

Where,
Y<< X >> = the certificate of user X issued by certification authority Y Y {I} = the signing of I by Y. It consists of I with an encrypted hash code appende

V = version of the certificate

SN = serial number of the certificate

AI = identifier of the algorithm used to sign the certificate CA = name of certificate authority
UCA = optional unique identifier of the CA

A = name of user A

UA = optional unique identifier of the user A

Ap = public key of user A

TA = period of validity of the certificate

Pseudo Random Numbers

- In terms of strength and security, random number generation is a vital part of all cryptography and spread spectrum systems.
- A weak random number generation may result in a compromised system in cryptographic or distributed spectrum systems.
- As such, a range of cryptographic or spread spectrum systems rely for improved protection on the generation of random and pseudorandom bits.
- Deterministic algorithms are used to construct a number series that is not statistically random.
- Good algorithms pass a number of tests of randomness.

True random numbers provide Pseudo -> Deterministic, reproducible, generated by a formula

$$x_n = f(x_{n-1}, x_{n-2}, \ldots)$$

Linear-Congruential Generators

- Discovered by D. H. Lehmer in 1951
- The residues of successive powers of a number have good randomness properties.

$$x_n = a^n \bmod m$$

Equivalently,

$$x_n = ax_{n-1} \bmod m$$

Where, a = multiplier
m = modulus

Lehmer's choices: a = 23 and m = 10^8+1

- Good for ENIAC, an 8-digit decimal machine.
- Generalization:

$$x_n = ax_{n-1} + b \bmod m$$

- Can be analyzed easily using the theory of congruences
- Mixed = both multiplication by a and addition of b

Blum Blum Shub Generator

From the iterative equation, use the least important bit:

Xi= X(i-1)^2 mod n

where n=p.q, and primes p,q=3 mod 4

- Unpredictable, Next-bit test passes
- Protection rests on the challenge of factoring N
- Is unpredictable given any run of bits
- Slow, because it is important to use very large numbers
- To slow down for cipher use, suitable for key generation

Pseudorandom Number Generators Using Block Cipher

- A block cipher can be used to produce random numbers for cryptographic apps,
- For the creation of session keys from master key
- CTR: Xi = EK[Vi]
- OFB: Xi = EK[Xi-1

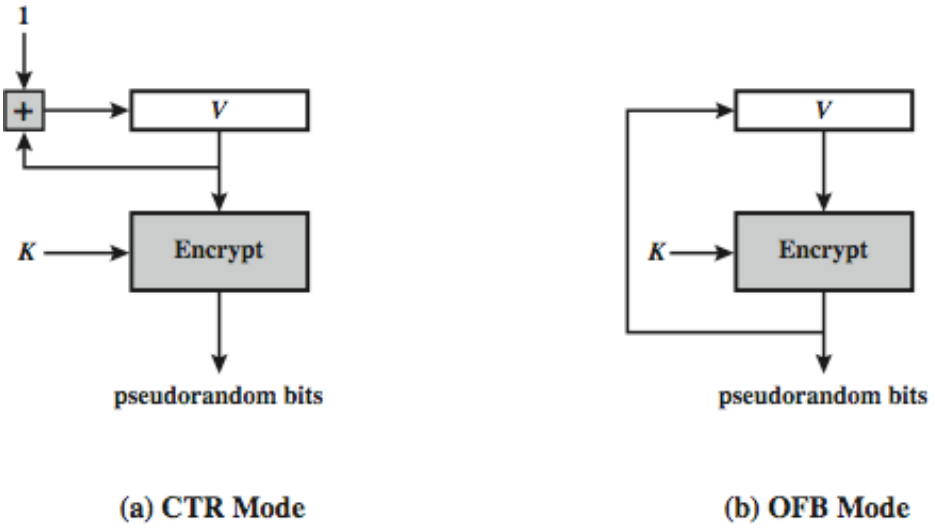


Figure 7.11 PRNG Mechanisms Using Block Cipher

Natural Random Noise

- The best source is real-world natural randomness.
- Check for a periodic but unpredictable event and track
- To do this, you usually need precise h/w.
  - E.g., radiation counters, radio noise, audio noise, thermal noise in diodes, leaky capacitors, mercury discharge tubes etc
- Beginning to see such h/w in new CPU's
- Problems with bias or unequal signal propagation
  - This must be compensated for while sampling, often by moving bits through a hash function.
  - Best to use just a few noisiest bits in every sample
  - RFC4086 recommends using multiple sources + hash

Stream Ciphers

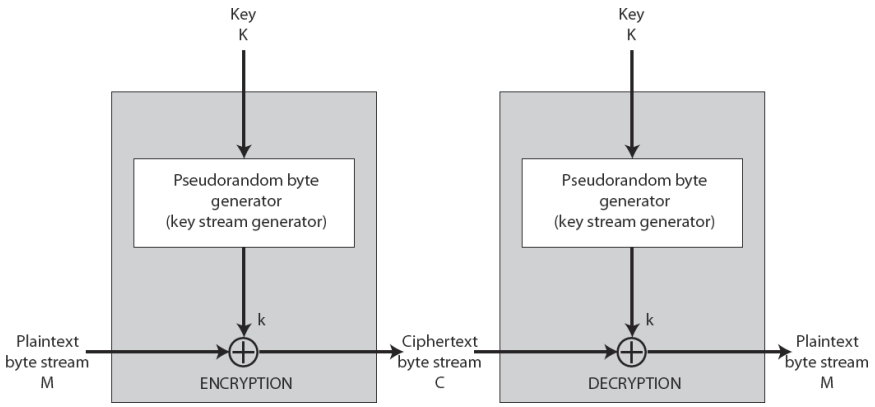


Figure 7.12 Stream Cipher



- A pseudo random keystream XOR'ed bit by bit with plaintext.

  Ci = Mi XOR StreamKeyi
- But stream key must never be reused, otherwise messages can be retrieved.

RC4

- A RSA DSI-owned proprietary cipher
- Another basic but powerful Ron Rivest concept
- Variable key size, byte-oriented stream cipher
- Widely used (web SSL/TLS, wireless WEP/WPA)
- Random permutation of all 8-bit values for main types
- Using this permutation to scramble input information to process a byte at a time.

RC4 Key Schedule

- Start with an array S of numbers: 0..255.
- Using the key to shuffle well and truly.
- S forms the cipher's internal state

```
For i = 0 to 255 do
        S[i] = i
        T[i] = K[i mod keylen])
j = 0
        for i = 0 to 255 do
        j = (j + S[i] + T[i]) (mod 256)
        swap (S[i], S[j])
```

RC4 Encryption

- Encryption continues to shuffle values from arrays.
- • The shuffled pair total selects the value of "stream key" from the permutation.
- XOR S[t] with next byte of message to en/decrypt

```
        i = j = 0
for each message byte Mi
        i = (i + 1) (mod 256)
        j = (j + S[i]) (mod 256)
        swap(S[i], S[j])
        t = (S[i] + S[j]) (mod 256)
        Ci = Mi XOR S [i]
```

